LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: N. Samhavya
Roll No.: A24126552099
Date: 29-08-2026

---

1. TITLE

Develop a Dynamic To-Do List Application Using JavaScript, DOM Manipulation and Event Listeners

---

2. AIM

To develop a dynamic **To-Do List application using JavaScript, DOM manipulation, and event listeners** that allows users to add, complete, and delete tasks dynamically from a webpage.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript DOM manipulation.
- To access HTML elements using JavaScript.
- To handle user actions using event listeners.
- To dynamically create new HTML elements.
- To add new tasks to a list dynamically.
- To implement a Complete button for each task.
- To visually indicate completed tasks.
- To implement a Delete button for each task.
- To remove individual tasks dynamically from the webpage.
- To display an appropriate message when no tasks are available.
- To handle the Enter key for adding tasks.
- To apply CSS styling to the To-Do List application.

---

4. THEORY

JavaScript is a programming language used to add dynamic behavior and interactivity to web pages. It can respond to user actions and modify webpage content without requiring the page to be reloaded.

The **DOM (Document Object Model)** represents an HTML document as a tree of objects. JavaScript can access, create, modify, and remove HTML elements using DOM methods. In this application, JavaScript is used to dynamically create task items and add them to an unordered list.

DOM Manipulation

DOM manipulation allows JavaScript to dynamically modify webpage content. In this experiment, methods such as getElementById(), createElement(), appendChild(), and remove() are used.

Event Listeners

The addEventListener() method is used to perform an action when a particular event occurs. In this application, click events are used for the **Add Task**, **Complete**, and **Delete** buttons.

Dynamic Task Creation

When the user enters a task and clicks the **Add Task** button, JavaScript creates a new <li> element dynamically and adds it to the task list. **Complete Functionality**

The Complete button uses classList.toggle() to add or remove the completed CSS class. The class applies a line-through effect to indicate that the task has been completed. **Delete Functionality**

The Delete button uses the remove() method to remove the corresponding task from the webpage.

Empty List Message

The program checks the number of child elements in the task list using:
taskList.children.length
If there are no tasks, the message **"No tasks available."** is displayed. The basic flow of the application is:
User Input → Add Task → DOM Element Creation → Task List → Complete/Delete → DOM Update

---

5. ALGORITHM / PROCEDURE

1. Create an HTML5 document.
2. Add a heading titled **My To-Do List**.
3. Create a text box for entering tasks.
4. Create an **Add Task** button.
5. Create an unordered list to display tasks.
6. Create a message to indicate that no tasks are available.
7. Create a JavaScript file.
8. Access the required HTML elements using getElementById().

9. Create a function to update the empty-list message.
10. Add a click event listener to the Add Task button.
11. Read the task entered by the user.
12. Remove unnecessary spaces using trim().
13. Check whether the task input is empty.
14. Display an alert if the input is empty.
15. Create a new <li> element dynamically.
16. Create a element for the task text.
17. Create a **Complete** button dynamically.
18. Add a click event listener to the Complete button.
19. Toggle the completed CSS class when the Complete button is clicked.
20. Create a **Delete** button dynamically.
21. Add a click event listener to the Delete button.
22. Remove the corresponding task when Delete is clicked.
23. Add the task text and buttons to the task item.
24. Add the task item to the task list.
25. Clear the input field after adding the task.
26. Update the empty-list message.
27. Add an event listener for the Enter key.
28. Allow the Enter key to add a task.
29. Call the empty-message function when the page initially loads.
30. Apply CSS styling to the webpage.
31. Test adding, completing, and deleting tasks.
32. Verify the final output.

---

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">    <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>My To-Do List</h1>
    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task">
      <button id="addTaskBtn">Add Task</button>
```

```
    </div>
    <ul id="taskList"></ul>
    <p      id="emptyMessage">
No tasks available.       </p>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

---

script.js

```
// Select required HTML elements using DOM methods const
taskInput = document.getElementById("taskInput"); const
addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");
// Function to update the empty-list message function
updateEmptyMessage() {
  // Check whether the task list contains any tasks
  if (taskList.children.length === 0) {
    emptyMessage.style.display = "block";
  } else {
    emptyMessage.style.display = "none";
  }
}
// Add a new task when the Add Task button is clicked
addTaskBtn.addEventListener("click", function () {
  // Get the task entered by the user
const taskText = taskInput.value.trim();
// Check whether the input is empty
  if (taskText === "") {
alert("Please enter a task.");
    return;
  }
  // Create a new list item
  const taskItem = document.createElement("li");
  // Add CSS class to the list item
taskItem.className = "task-item";    //
Create element for task text
  const task = document.createElement("span");
  task.className = "task-text";
task.textContent = taskText;    //
Create Complete button
```

```
   const completeBtn = document.createElement("button");
completeBtn.className = "complete-btn";
completeBtn.textContent = "Complete";    // Add click
event to Complete button
   completeBtn.addEventListener("click", function () {
      // Add or remove the completed class
task.classList.toggle("completed");
   });
   // Create Delete button     const deleteBtn =
document.createElement("button");
deleteBtn.className = "delete-btn";
deleteBtn.textContent = "Delete";    // Add click event
to Delete button     deleteBtn.addEventListener("click",
function () {        // Remove the selected task from the
webpage        taskItem.remove();
      // Update empty-list message
      updateEmptyMessage();
   });
   // Add elements to the task item
taskItem.appendChild(task);
taskItem.appendChild(completeBtn);
taskItem.appendChild(deleteBtn);    //
Add the task item to the task list
taskList.appendChild(taskItem);
   // Clear the input box
taskInput.value = "";    //
Update empty-list message
   updateEmptyMessage();
});
// Allow the Enter key to add a task
taskInput.addEventListener("keypress", function (event) {

   if (event.key === "Enter") {
      addTaskBtn.click();
   }

});
// Display the empty message when the page first loads
updateEmptyMessage();
```

---

style.css

```
body {
font-family: Arial, sans-serif;
 background-color: #f2f4f7;
```

```
margin: 0;
padding: 40px;
}
.container {
width:600px;
max-width:90%;
margin:auto;
padding:25px;
background-color:
rgb(242, 237, 237);
border-
radius:10px;
 box-shadow: 0 4px 12px rgb(27, 109, 153);
}
h1 {
text-align: center;
color: #000000;
}
.input-section {
display: flex;
 gap: 10px;
}
#taskInput {
 flex: 1;
 padding: 12px;
border: 1px solid #46134b;
border-radius: 5px;
font-size: 15px;
}
#addTaskBtn {
padding: 12px 18px;
background-color: #8f1737;
color: rgb(240, 232, 232);
border: none;
border-radius: 5px;     cursor:
pointer;
}
#addTaskBtn:hover {     background-
color: #22705f;
}
```

```
#taskList {     list-
style: none;
padding: 0;
margin-top: 25px;
}
.task-item {
display: flex;
align-items: center;
 justify-content: space-between;
padding: 12px;
 margin-bottom: 10px;
background-color: #ac97c0;
border: 1px solid #049b2f;
 border-radius: 5px;
}
.task-text {
flex: 1;
font-size: 16px;
}
.completed {
text-decoration: line-through;
color: rgb(0, 0, 0);
}
.complete-btn {
margin-left: 10px;
padding: 8px 12px;
background-color: #08530d;
color: white;
border: none;
border-radius: 4px;
 cursor: pointer;
}
.delete-btn {
margin-left: 8px;
padding: 8px 12px;
background-color: #c71010fb;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
}
```

```
.complete-btn:hover {
 background-color: #e20707;
}
.delete-btn:hover {
background-color: #a82c2c;
}
#emptyMessage {
   text-align: center;
color: #0d5444;
font-style: italic;
}
```

---

7. CODE EXPLANATION

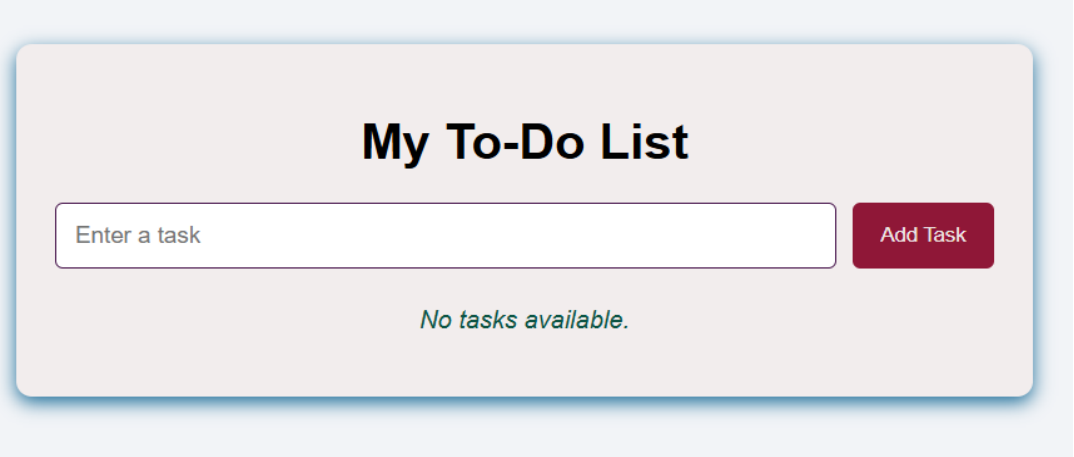
Code	Explanation
getElementById()	Selects an HTML element using its ID.
addEventListener()	Attaches an event handler to an HTML element.
taskInput.value	Retrieves the task entered by the user.
trim()	Removes unnecessary spaces from the beginning and end of the input.
alert()	Displays a message to the user.
createElement()	Dynamically creates a new HTML element.
className	Assigns a CSS class to an element.
textContent	Sets the text content of an element.
appendChild()	Adds a dynamically created element to another element.
classList.toggle()	Adds or removes the specified CSS class.
remove()	Removes an HTML element from the webpage.
children.length	Determines the number of child elements in the task list.
style.display	Controls whether the empty-list message is displayed.
keypress	Detects keyboard input events.
event.key	Identifies the key pressed by the user.
.completed	CSS class used to visually indicate a completed task.
:hover	Applies CSS styling when the pointer moves over an element.

---

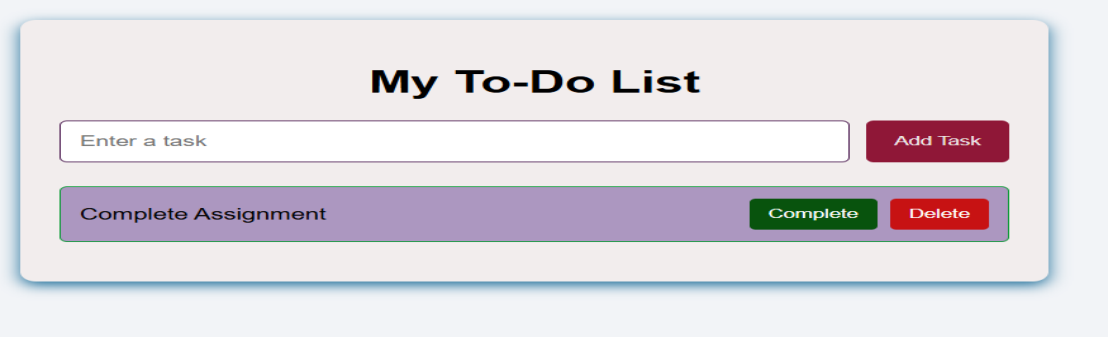
8. TEST CASES AND EXECUTION DATA

Test Case	Test / Input	Expected Result	Actual Result	Status
TC-01	Open the To-Do List webpage	To-Do List interface should be displayed with input field and Add Task button	Interface displayed correctly	**Pass**
TC-02	Enter **"Complete Assignment"** and click **Add Task**	The task should be added to the task list	Task added successfully	**Pass**
TC-03	Add **"Prepare for Quiz"** and **"Practice JavaScript"**	Both tasks should appear in the task list	Both tasks displayed correctly	**Pass**
TC-04	Click **Complete** for a task	The selected task should be displayed with a line-through effect	Task marked as completed successfully	**Pass**
TC-05	Click **Delete** for a task	The selected task should be removed from the list	Task removed successfully	**Pass**
TC-06	Delete all tasks from the list	**"No tasks available."** message should be displayed	Message displayed correctly	**Pass**
TC-07	Click **Add Task** without entering a task	Alert should request the user to enter a task	Alert displayed correctly	**Pass**
TC-08	Enter a task and press **Enter**	The task should be added to the list	Task added successfully	**Pass**

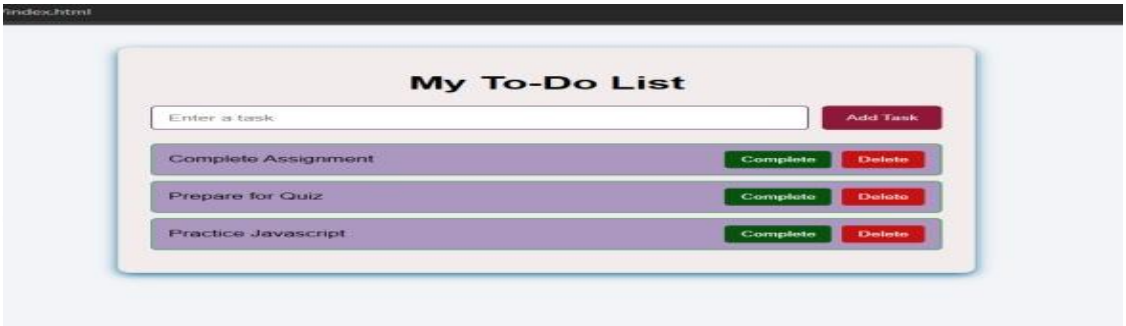
TC-01 Output:



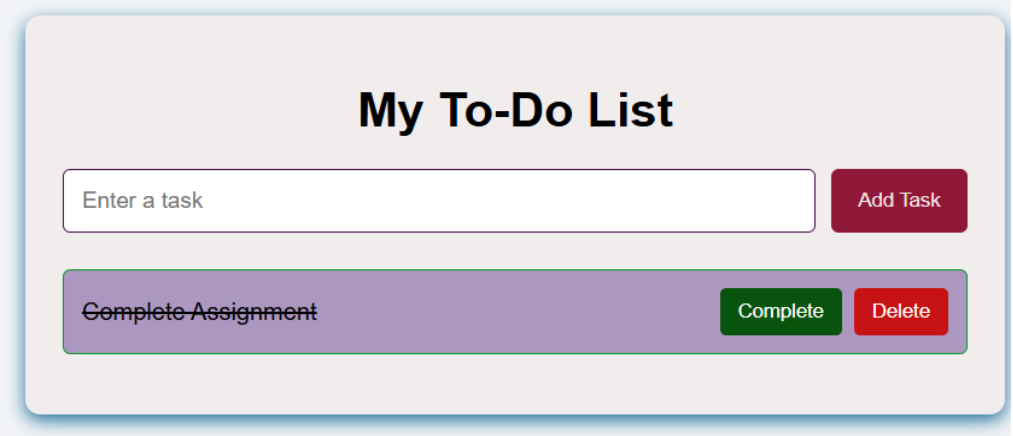
TC-02 Output:



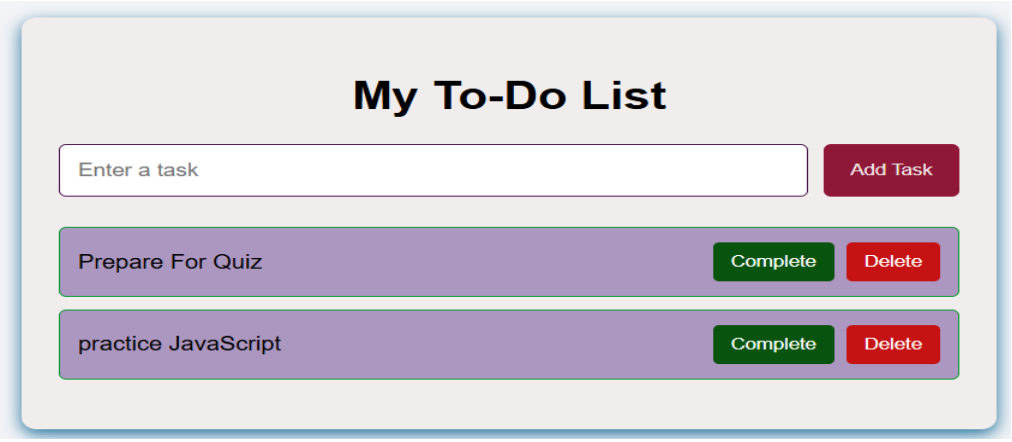
TC-03 Output:



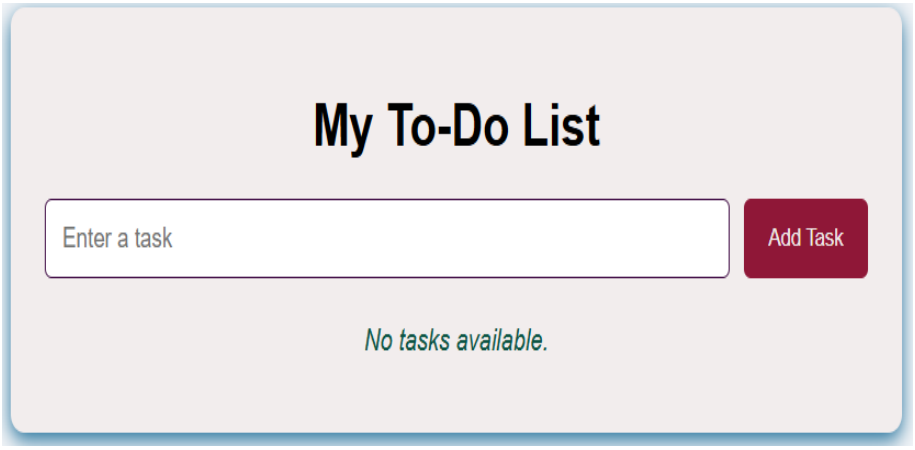
TC-04 Output:



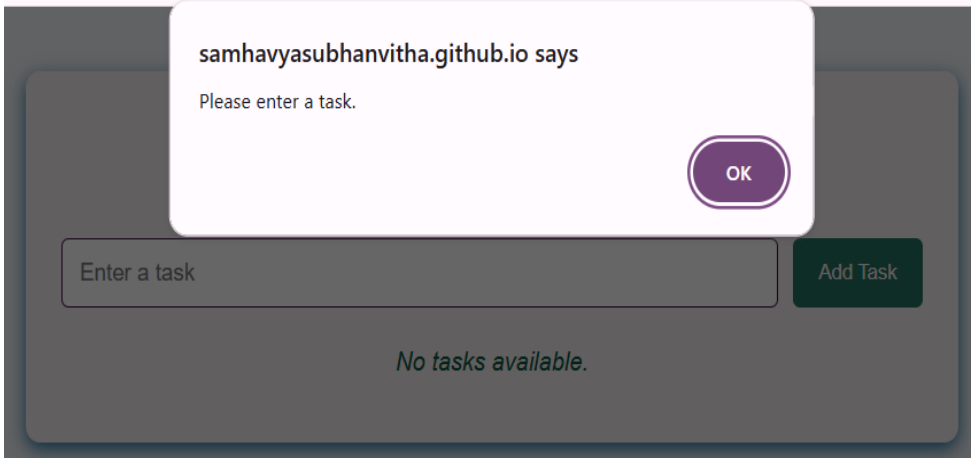
TC-05 Output:



TC-06 Output:



TC-07 Output:



---

9. OUTPUT

Output : To-Do List After Adding Tasks

The following tasks are entered during execution:

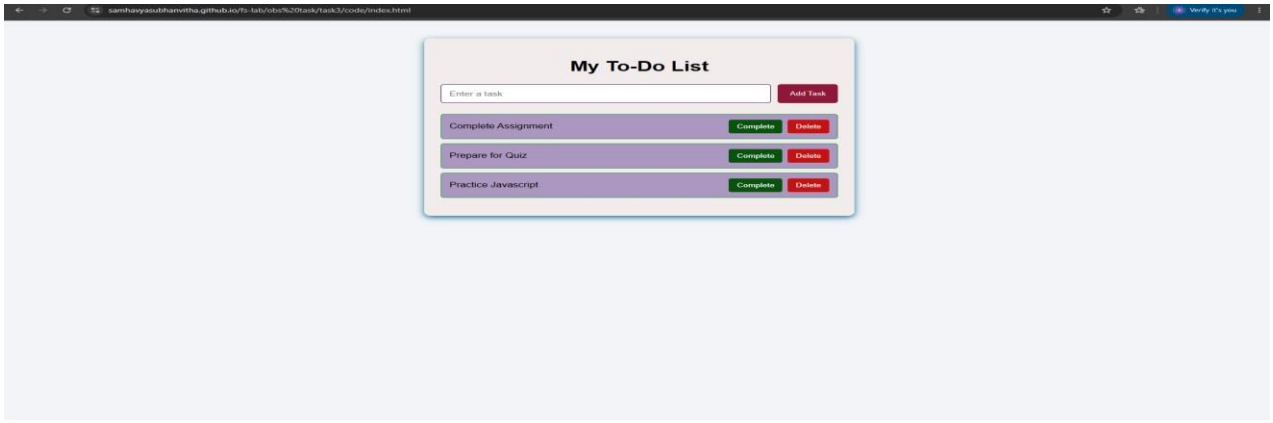
- Complete Assignment
- Prepare for Quiz Practice JavaScript The

resulting webpage displays:

Complete Assignment — Complete — Delete
Prepare for Quiz — Complete — Delete
Practice JavaScript — Complete — Delete

When the **Complete** button is clicked, the corresponding task is displayed with a **linethrough effect**.

When the **Delete** button is clicked, the selected task is removed from the list.



---

10. RESULT

Thus, a **dynamic To-Do List application** was successfully developed using **JavaScript, DOM manipulation, and event listeners**. Tasks were dynamically added to the list, individual tasks could be marked as completed, and tasks could be deleted from the webpage. The application also displayed an appropriate message when the task list was empty and was successfully verified using multiple test cases.